

Arquitecturas Modernas en la Nube

G. Monzón Flores

7690-20-20745 Universidad Mariano Gálvez

7690 Seminario de Tecnología de Información

gmonzonf1@miumg.edu.gt

Resumen

El presente artículo analiza la relación entre la orquestación de servidores, Kubernetes, la arquitectura de microservicios, OAuth 2.0 y los principios de las aplicaciones de doce factores para implementar servicios en la nube de forma segura y sostenible. El objetivo fue identificar cómo estas prácticas complementan el despliegue de aplicaciones distribuidas y qué decisiones evitan que la adopción de tecnología aumente innecesariamente la complejidad operativa. Para ello, se revisó documentación técnica actual de Kubernetes, OAuth, Microsoft y la metodología Twelve-Factor App. El análisis mostró que los microservicios permiten separar responsabilidades y escalar componentes de forma independiente, pero también incrementan la necesidad de automatizar despliegues, observabilidad, configuración y comunicación entre servicios. Kubernetes aporta mecanismos declarativos para administrar contenedores, reemplazar instancias defectuosas y ajustar la capacidad, mientras que OAuth 2.0 permite delegar la autorización mediante tokens con permisos definidos. Los principios de doce factores complementan este modelo al separar configuración y código, mantener procesos sin estado y tratar las dependencias externas como recursos conectables. Se concluye que una implementación en la nube no depende únicamente de migrar servidores, sino de diseñar servicios operables, con controles de acceso, automatización y responsabilidades técnicas claramente definidas.

Palabras clave: microservicios, Kubernetes, OAuth 2.0, computación en la nube, doce factores

Desarrollo del tema

Las aplicaciones modernas suelen atender usuarios desde distintos dispositivos, integrarse con plataformas externas y procesar una cantidad variable de solicitudes. En este contexto, alojar todo el sistema en un único servidor puede ser suficiente para una aplicación pequeña, pero se vuelve limitado cuando se requiere desplegar cambios frecuentes, escalar una función específica o recuperarse rápidamente de una falla. La implementación de servicios en la nube busca aprovechar recursos que pueden aprovisionarse y adaptarse con mayor rapidez, aunque esta flexibilidad exige una arquitectura y una operación más disciplinadas.

La orquestación de servidores consiste en coordinar de manera automatizada la configuración, ejecución, disponibilidad y escalamiento de recursos de cómputo. Su propósito no es reemplazar todo el trabajo de administración, sino expresar el estado deseado de una aplicación y permitir que la plataforma lo mantenga. En lugar de iniciar manualmente varias instancias, copiar configuraciones y revisar una por una si permanecen activas, el equipo declara cuántas réplicas necesita, qué recursos pueden consumir y cómo deben exponerse. La plataforma compara esa definición con el estado real y ejecuta acciones para reducir la diferencia.

Kubernetes es una plataforma de código abierto para administrar cargas de trabajo y servicios en contenedores mediante configuración declarativa y automatización (Kubernetes, 2026). Los contenedores empaquetan una aplicación con sus dependencias de ejecución, de manera que el mismo artefacto puede usarse en desarrollo, pruebas y producción. Kubernetes no sustituye el diseño de una aplicación ni convierte automáticamente un sistema en escalable; su aporte es administrar el ciclo de vida de los contenedores. Por ejemplo, puede programar réplicas en nodos disponibles, reiniciar un contenedor que falla, distribuir solicitudes entre instancias y aplicar actualizaciones graduales.

Los objetos de Kubernetes hacen visible esa intención operativa. Un *Pod* representa la unidad que ejecuta uno o más contenedores estrechamente relacionados; un *Deployment* declara el número de réplicas y permite realizar actualizaciones controladas; un *Service* proporciona una dirección estable para acceder a una aplicación aunque sus instancias cambien; y un *Ingress* o puerta de enlace de API puede gestionar la entrada de tráfico externo. Esta abstracción evita depender de la dirección de un servidor específico, pero requiere definir límites de recursos, verificaciones de salud y políticas de acceso. De lo contrario, el clúster puede amplificar una mala configuración a gran escala. Kubernetes recomienda declarar solicitudes y límites de recursos para que el programador pueda asignar adecuadamente las cargas de trabajo (Kubernetes, 2026).

La arquitectura de microservicios divide una aplicación en servicios pequeños, cada uno orientado a una capacidad de negocio concreta. Una plataforma de comercio electrónico podría separar el catálogo, pagos, inventario y notificaciones. Esta separación permite desplegar o escalar el catálogo sin publicar nuevamente el módulo de pagos. Sin embargo, también introduce comunicaciones de red, contratos entre interfaces, fallos parciales, monitoreo distribuido y más componentes. Un monolito modular puede ser mejor cuando el producto es pequeño o el equipo aún no puede operar un sistema distribuido.

Por esa razón, el valor de los microservicios no se mide por la cantidad de servicios creados, sino por el nivel de independencia que aportan. Cada servicio debe poseer una responsabilidad clara, exponer interfaces documentadas y evitar compartir directamente su base de datos con los demás. La comunicación puede realizarse mediante API o mensajería, según el tipo de proceso. Una petición de consulta normalmente requiere respuesta inmediata, mientras que una notificación o una actualización no crítica puede procesarse de manera asíncrona. Esta distinción mejora la resiliencia, pero demanda trazabilidad: ante un error, el equipo debe poder saber qué servicio recibió la solicitud, cuál dependía de otro y en qué punto se produjo la falla.

La seguridad adquiere otra dimensión cuando varios servicios se comunican entre sí. No es correcto asumir que una solicitud interna es segura solo porque proviene de la red de la organización. OAuth 2.0 es un estándar de autorización que permite a una aplicación obtener acceso limitado a un recurso sin revelar la contraseña del usuario al servicio consumidor (OAuth, 2026). Sus participantes principales son el propietario del recurso, el cliente, el servidor de autorización y el servidor de recursos. El servidor de autorización emite un token de acceso después de validar la autorización; posteriormente, el servicio que protege la información valida el token y sus permisos antes de responder.

OAuth 2.0 no equivale por sí mismo a autenticación ni a control completo de identidad. Su función principal es autorizar qué puede hacer un cliente mediante alcances o *scopes*. Para aplicaciones web y móviles, el flujo de código de autorización con PKCE es una opción apropiada cuando existe un usuario que concede permisos; para servicios sin interacción de usuario, el flujo de credenciales de cliente permite identificar la aplicación que solicita el recurso. Los tokens deben tener tiempo de vida limitado, audiencia definida y privilegios mínimos. En una arquitectura de microservicios, una puerta de enlace puede validar inicialmente el token, pero cada servicio que procesa datos sensibles debe verificar la autorización relevante y no confiar ciegamente en la petición anterior (Microsoft, 2026). Los principios de la metodología Twelve-Factor App ayudan a que estos servicios sean más fáciles de desplegar y reemplazar. Entre los principios más relevantes se encuentran mantener un solo código fuente bajo control de versiones, declarar explícitamente las dependencias, separar la configuración del código y tratar bases de datos, colas y almacenamiento como recursos conectables. La configuración incluye direcciones de servicios, credenciales y valores que cambian entre desarrollo y producción; por eso no debe quedar escrita como una constante dentro del repositorio. La metodología propone

almacenar dichos valores en el entorno, de forma que el mismo código pueda desplegarse en distintos contextos sin exponer secretos (Wiggins, 2017).

Otro principio esencial es ejecutar los servicios como procesos sin estado. La información que debe persistir se almacena en una base de datos, una caché gestionada o una cola, no en el disco local ni en la memoria de una instancia particular. Esto permite que Kubernetes agregue o reemplace réplicas sin perder la coherencia de la aplicación. De la misma manera, los procesos deben iniciar con rapidez y finalizar ordenadamente; así pueden participar en despliegues graduales y recuperarse cuando una verificación de salud detecta un problema. Los registros se tratan como flujos de eventos y se centralizan, en lugar de depender de revisar manualmente archivos dentro de cada contenedor. Una implementación responsable en la nube integra estas prácticas en una ruta de entrega continua. El código se construye como un artefacto reproducible, se despliega con configuración externa y se monitorean métricas, registros y trazas. Las identidades de los servicios y los secretos deben administrarse mediante mecanismos de la plataforma, no copiándose en archivos de despliegue. También son necesarios respaldos, límites de costo y planes de recuperación. Migrar a contenedores sin estas decisiones puede producir el mismo sistema frágil de antes, solo repartido en más máquinas.

Observaciones y comentarios

Durante el análisis se observó que Kubernetes y los microservicios suelen presentarse como requisitos inevitables para cualquier aplicación en la nube. Sin embargo, estas tecnologías resuelven problemas concretos y también trasladan al equipo nuevas responsabilidades de seguridad, monitoreo y operación. Como mejora, antes de dividir un sistema se debería evaluar qué componentes realmente necesitan escalar o desplegarse de manera independiente. Asimismo, OAuth 2.0 debe incorporarse junto con una política de permisos y gestión de secretos, ya que emitir tokens sin limitar sus alcances solo cambia la forma de una posible brecha.

Conclusiones

1. La orquestación permite administrar el estado deseado de aplicaciones en contenedores y reduce las tareas manuales de despliegue, recuperación y escalamiento.
2. Kubernetes facilita operar microservicios, pero requiere configuraciones explícitas de recursos, salud, redes y seguridad para aportar valor real.
3. Los microservicios son apropiados cuando la independencia de funciones compensa la complejidad de la comunicación y operación distribuida.
4. OAuth 2.0 permite delegar la autorización mediante tokens y alcances, pero debe aplicarse con validación, privilegio mínimo y protección de secretos.
5. Los principios de doce factores conectan el desarrollo con la operación en la nube al promover servicios configurables, sin estado, observables y reemplazables.

Referencias

Kubernetes. (2026). *Kubernetes documentation*. <https://kubernetes.io/docs/home/>

Microsoft. (2026). *Microservices architecture on Azure Kubernetes Service*. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/reference-architectures/containers/aks-microservices/aks-microservices>

OAuth. (2026). *OAuth 2.0*. <https://oauth.net/2/>

Wiggins, A. (2017). *The Twelve-Factor App*. <https://12factor.net/es/>

Repositorio <https://github.com/DaniSaurioM/ForosAcademicos-Seminario>